Rekonstrukcija ravninske površine iz zašumljenega 3D-oblaka točk z metodo PCA

Matematična izpeljava, implementacija v C++ in statistično vrednotenje

Raziskovalno-tehnični članek | 6. november 2026

Povzetek. Prispevek obravnava ocenjevanje ravninske površine iz tridimenzionalnih meritev z aditivnim Gaussovim šumom. Referenčni oblak 1681 točk je generiran na kvadratni ravnini, transformiran v prostor in obremenjen z izotropnim šumom. Lega in orientacija ravnine sta določeni z analizo glavnih komponent (PCA), tj. z lastnim razcepom kovariančne matrike centriranih točk. Prikazana je izpeljava ekvivalentnosti PCA in ortogonalnih najmanjših kvadratov, nato pa opisana implementacija v programskem jeziku C++20 s knjižnico Eigen. Vrednotenje zajema en prikaz rekonstrukcije in 600 ponovitev Monte Carlo pri šestih jakostih šuma. Pri standardnem odklonu $\sigma = 100\,\mathrm{mm}$ znašajo povprečna kotna napaka $0{,}2971^\circ$, povprečni RMSE $99{,}65\,\mathrm{mm}$ in povprečna napaka težišča $4{,}08\,\mathrm{mm}$. Rezultati so skladni s pričakovanim statističnim obnašanjem ocene ravnine.

Ključne besede: PCA, oblak točk, kovariančna matrika, lastni razcep, ravnina, Gaussov šum, Monte Carlo, C++/Eigen.

1 Uvod

Rekonstrukcija geometrijskih struktur iz oblaka točk je osnovna operacija v robotskem zaznavanju, tridimenzionalnem kartiranju in obdelavi meritev globinskih senzorjev. Pri ravninskih površinah želimo iz množice točk oceniti orientacijo in položaj ravnine. Problem otežuje merilni šum: tudi če točke izvirajo z idealno ravne površine, v prostoru ne ležijo na isti ravnini. Namesto interpolacije posameznih meritev zato uporabimo statistično oceno geometrije celotnega oblaka.

Metoda analize glavnih komponent (angl. *principal component analysis*, PCA) določi smeri, v katerih so podatki najbolj oziroma najmanj razpršeni [1]. Za približno ravninski oblak je smer najmanjše razpršenosti normala najbolje prilegajoče se ravnine. Prednost metode je enostavna in deterministična numerična rešitev; njena glavna omejitev je občutljivost na točke, ki ne pripadajo obravnavani površini. Cilj prispevka je izpeljati metodo, opisati njeno implementacijo in eksperimentalno preveriti vpliv šuma na natančnost.

2 Generiranje referenčnega oblaka in model meritev

2.1 Vzorčenje in prostorska transformacija

V koordinatnem sistemu XY ustvarimo kvadratno mrežo točk

$$\boldsymbol{p}_{ij}^{(0)} = \begin{bmatrix} i\Delta & j\Delta & 0 \end{bmatrix}^T, \quad i,j = 0,\dots,N, \qquad M = (N+1)^2. \tag{1}$$

Pri $\Delta = 0{,}05\,\mathrm{m}$ in $N = 40$ je stranica kvadrata dolga $N\Delta = 2\,\mathrm{m}$, število točk pa $M = 1681$. Njegova začetna normala je $\boldsymbol{n}_0 = (0,0,1)^T$. Da preverimo rekonstrukcijo pri poljubnejši orientaciji, ravnino zavrtimo

Tabela 1. Parametri referenčne geometrije in Monte Carlo poskusa.

Parameter	Vrednost	Parameter	Vrednost
Velikost površine	$2 \times 2\,\mathrm{m}^2$	Število točk	1681
Korak mreže	$0{,}05$ m	Rotaciji	$6{,}1^\circ, 12{,}3^\circ$
Translacija	$(0{,}02, 0{,}13, 0{,}07)$ m	Generator	MT19937, seme 42
σ	0, 5, 10, 30, 50, 100 mm	Ponovitve	100 na vrednost σ

okoli osi X in Y ter premaknemo:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix}, \qquad R_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix}, \tag{2}$$

$$\boldsymbol{p}_i^{\mathrm{ref}} = R\boldsymbol{p}_i^{(0)} + \boldsymbol{t}, \qquad R = R_y(\beta)R_x(\alpha). \tag{3}$$

Uporabljena kota sta $\alpha = 6{,}1^\circ$, $\beta = 12{,}3^\circ$, translacija pa $\boldsymbol{t} = (0{,}02,\ 0{,}13,\ 0{,}07)^T$ m. Vrstni red rotacij ni poljuben: pri zapisu s stolpčnimi vektorji najprej deluje R_x, nato R_y. Ker je $R^TR = I$, toga transformacija ohranja razdalje. Referenčno normalo po rotaciji poznamo analitično kot $\boldsymbol{n}_{\mathrm{ref}} = R\boldsymbol{n}_0$; translacija normale ne spremeni. Transformacijo bi lahko ekvivalentno zapisali s homogeno matriko $T \in SE(3)$, vendar je za to nalogo zapis $R\boldsymbol{p} + \boldsymbol{t}$ neposrednejši.

2.2 Gaussov model šuma

Vsaki referenčni točki dodamo neodvisen tridimenzionalni šum

$$\boldsymbol{p}_i^{\mathrm{meas}} = \boldsymbol{p}_i^{\mathrm{ref}} + \boldsymbol{\varepsilon}_i, \qquad \boldsymbol{\varepsilon}_i \sim \mathcal{N}(\boldsymbol{0}, \sigma^2 I_3). \tag{4}$$

Predpostavka $\sigma^2 I_3$ pomeni enako varianco v smereh X, Y in Z ter ničelne medsebojne kovariance; šum je torej izotropen. Standardni odklon σ ni največji dovoljen odmik, temveč parameter porazdelitve posamezne koordinate. Tak model omogoča preverljiv statistični eksperiment, ni pa popoln opis realnega RGB-D senzorja, kjer je merilna negotovost lahko odvisna od globine in vpadnega kota.

3 Ocena ravnine z analizo glavnih komponent

3.1 Težišče in centriranje

Izmerjeni oblak označimo z $\{\boldsymbol{p}_i\}_{i=1}^{M}$; indeks meas v nadaljevanju izpustimo. Njegovo težišče je aritmetično povprečje, centrirani vektorji pa odmiki posameznih točk od težišča:

$$\bar{\boldsymbol{p}} = \frac{1}{M}\sum_{i=1}^{M} \boldsymbol{p}_i, \qquad \boldsymbol{q}_i = \boldsymbol{p}_i - \bar{\boldsymbol{p}}, \qquad \sum_{i=1}^{M} \boldsymbol{q}_i = \boldsymbol{0}. \tag{5}$$

Centriranje je bistveno, ker želimo analizirati obliko in orientacijo oblaka, ne njegove absolutne lege. Če vse točke premaknemo za enak vektor, ostanejo centrirani odmiki nespremenjeni. Posledično bo tudi ocenjena normala translacijsko invariantna.

3.2 Kovariančna matrika in geometrijski pomen

Razpršenost točk opiše kovariančna matrika

$$C = \frac{1}{M}\sum_{i=1}^{M} \boldsymbol{q}_i \boldsymbol{q}_i^T = \begin{bmatrix} C_{xx} & C_{xy} & C_{xz} \\ C_{xy} & C_{yy} & C_{yz} \\ C_{xz} & C_{yz} & C_{zz} \end{bmatrix}. \tag{6}$$

Produkt $\boldsymbol{q}_i \boldsymbol{q}_i^T$ je *zunanji produkt*: iz vektorja velikosti 3×1 in njegove transponirane oblike 1×3 nastane matrika 3×3. Diagonalni členi predstavljajo variance posameznih koordinat, nediagonalni pa kovariance oziroma skupno linearno spreminjanje koordinat. Matrika C je simetrična in pozitivno semidefinitna, saj za poljuben vektor $\boldsymbol{u}$ velja

$$\boldsymbol{u}^T C \boldsymbol{u} = \frac{1}{M}\sum_{i=1}^{M} (\boldsymbol{u}^T \boldsymbol{q}_i)^2 \geq 0. \tag{7}$$

Izraz na desni je povprečna kvadratna projekcija centriranih točk v smeri $\boldsymbol{u}$. Kovariančna matrika torej omogoča merjenje razpršenosti v katerikoli prostorski smeri.

3.3 Lastni razcep in izbira normale

Ker je C simetrična, obstaja ortonormiran nabor lastnih vektorjev $\boldsymbol{v}_k$ z realnimi nenegativnimi lastnimi vrednostmi λ_k:

$$C\boldsymbol{v}_k = \lambda_k \boldsymbol{v}_k, \qquad C = V \Lambda V^T, \qquad 0 \leq \lambda_1 \leq \lambda_2 \leq \lambda_3. \tag{8}$$

Lastni vektor je smer, pripadajoča lastna vrednost pa varianca točk v tej smeri. Pri skoraj ravninski množici sta dve smeri razpršenosti znotraj ravnine izraziti, smer pravokotno nanjo pa ima najmanjšo varianco. Zato je ocena enotske normale

$$\boxed{\hat{\boldsymbol{n}} = \boldsymbol{v}_1, \qquad \|\hat{\boldsymbol{n}}\|_2 = 1.} \tag{9}$$

Predznak lastnega vektorja je poljuben: $\hat{\boldsymbol{n}}$ in $-\hat{\boldsymbol{n}}$ opisujeta isto neorientirano ravnino. PCA zato sama po sebi ne določa, na katero stran površine naj kaže normala.

3.4 Povezava z ortogonalnimi najmanjšimi kvadrati

Razlog, da PCA vrne najbolje prilegajočo se ravnino, je mogoče pokazati brez dodatnih predpostavk o Gaussovi porazdelitvi. Ravnino zapišemo kot $\boldsymbol{n}^T \boldsymbol{p} + d = 0$, kjer je $\|\boldsymbol{n}\|_2 = 1$. Ta pogoj zagotavlja, da je $e_i = \boldsymbol{n}^T \boldsymbol{p}_i + d$ predznačena *pravokotna razdalja* točke od ravnine. Iščemo minimum povprečja kvadratov teh razdalj:

$$\min_{\|\boldsymbol{n}\|_2 = 1,\ d} J(\boldsymbol{n}, d), \qquad J(\boldsymbol{n}, d) = \frac{1}{M}\sum_{i=1}^{M} (\boldsymbol{n}^T \boldsymbol{p}_i + d)^2. \tag{10}$$

Za dano normalo odvajamo kriterij po d in odvod izenačimo z nič. Dobimo $d = -\boldsymbol{n}^T \bar{\boldsymbol{p}}$, zato mora optimalna ravnina potekati skozi težišče. Če ta izraz vstavimo nazaj v (10), sledi

$$J(\boldsymbol{n}) = \frac{1}{M}\sum_{i=1}^{M} (\boldsymbol{n}^T \boldsymbol{q}_i)^2 = \boldsymbol{n}^T C \boldsymbol{n}. \tag{11}$$

Po Rayleigh–Ritzovem izreku doseže izraz $\boldsymbol{n}^T C \boldsymbol{n}$ pri enotskih vektorjih najmanjšo vrednost λ_1, in sicer v smeri $\boldsymbol{v}_1$ [2]. S tem dobimo končno oceno ravnine

$$\boxed{\hat{\boldsymbol{n}} = \boldsymbol{v}_1, \qquad \hat{d} = -\hat{\boldsymbol{n}}^T \bar{\boldsymbol{p}}, \qquad \hat{\boldsymbol{n}}^T \boldsymbol{p} + \hat{d} = 0.} \tag{12}$$

PCA je torej za eno ravnino in kriterij pravokotnih kvadratnih odmikov globalna rešitev *ortogonalnih najmanjših kvadratov*. Optimalnost velja za ta kriterij; ni pa jamstvo pravilne segmentacije pri osamelcih ali več različnih površinah.

4 Metrike kakovosti rekonstrukcije

Orientacijo vrednotimo s kotom med referenčno in ocenjeno normalo. Ker sta smeri $\hat{\boldsymbol{n}}$ in $-\hat{\boldsymbol{n}}$ ekvivalentni, v skalarnem produktu uporabimo absolutno vrednost:

$$\theta = \frac{180}{\pi} \arccos\!\left(\mathrm{clip}\left(\left|\boldsymbol{n}_{\mathrm{ref}}^{T}\hat{\boldsymbol{n}}\right|, 0, 1\right)\right). \tag{13}$$

Kot θ je izražen v stopinjah. Operacija clip prepreči numerično napako funkcije arccos, če zaokroževanje povzroči vrednost nekoliko nad 1.

Prileganje ocenjene ravnine izmerjenim točkam merimo s korenom povprečne kvadratne napake (RMSE), izračunane iz predznačenih pravokotnih odmikov:

$$\mathrm{RMSE} = \sqrt{\frac{1}{M}\sum_{i=1}^{M}(\hat{\boldsymbol{n}}^{T}\boldsymbol{p}_i + \hat{d})^2} = \sqrt{\lambda_1}. \tag{14}$$

Zadnja enakost velja natančno za minimizacijski kriterij (10) in kovariančno matriko (6) z deljenjem z M. RMSE meri odstopanja *točk od ocenjene ravnine*, zato ga ne smemo enačiti z napako položaja same ravnine glede na referenco.

Napako težišča določimo z

$$e_c = \|\bar{\boldsymbol{p}}_{\mathrm{meas}} - \bar{\boldsymbol{p}}_{\mathrm{ref}}\|_2. \tag{15}$$

Pri neodvisnem izotropnem Gaussovem šumu ima vsaka koordinata napake težišča varianco σ^2/M. Ker je napaka e_c norma trirazsežnega normalnega vektorja, njena pričakovana vrednost znaša

$$\mathbb{E}[e_c] = 2\sqrt{\frac{2}{\pi}}\,\frac{\sigma}{\sqrt{M}}. \tag{16}$$

Pri $M = 1681$ in $\sigma = 100\,\mathrm{mm}$ dobimo $\mathbb{E}[e_c] \approx 3{,}89\,\mathrm{mm}$. Teorija zato napoveduje bistveno manjšo napako težišča, kot je šum posamezne meritve.

5 Programsko ogrodje in implementacija

Algoritem je implementiran v jeziku C++20; knjižnica Eigen skrbi za linearno algebro, standardna knjižnica `<random>` za simulacijo šuma, CSV-izvoz pa za povezavo s Pythonovo vizualizacijo. Celotno zaporedje računa ostane v eni datoteki `main.cpp`. Tabela 2 povzema vse razvite funkcije in njihove naloge, spodnji izseki pa pojasnjujejo matematično najpomembnejše operacije.

5.1 Generiranje, transformacija in šum

Funkcija `PointGen` shrani koordinate $(i\Delta, j\Delta, 0)$ v `std::vector<Eigen::Vector3d>`. Množenje z rotacijsko matriko je zapisano enako kot v enačbi (3), uporaba `Eigen::AngleAxisd` pa omogoča sestavo rotacij okoli izbranih osi. Funkcija `AddGausNoise` uporablja isto porazdelitev za vsako koordinato, vendar ob vsakem klicu vzorči novo naključno vrednost:

```
void TransformPoints(std::vector<Eigen::Vector3d>& points,
                     const Eigen::Matrix3d& R,
                     const Eigen::Vector3d& t) {
```

Tabela 2. Opis funkcij programske implementacije in njihova povezava z matematičnim postopkom.

Funkcija / struktura	Naloga in razlog za uporabo
`PointGen(step,N)`	Generira pravilno vzorčeno ravnino XY s $(N+1)^2$ točkami; `reserve` zmanjša morebitne ponovne alokacije.
`TransformPoints(…)`	Na vsaki točki izvede $R\boldsymbol{p}+\boldsymbol{t}$; matrika in translacija sta podani po konstantni referenci.
`SaveCSV(…)`	Zapiše koordinate x, y, z z glavo CSV; podatke loči od prikaza in omogoča preverljivost rezultatov.
`AddGausNoise(…)`	Vsaki koordinati doda vzorec $\mathcal{N}(0, \sigma^2)$; `mt19937` se posreduje po referenci.
`CalcCentroid(…)`	Izračuna $\bar{\boldsymbol{p}} = M^{-1} \sum \boldsymbol{p}_i$; funkcija določi tudi točko, skozi katero poteka optimalna ravnina.
`CalcCovariance(…)`	Sešteva zunanje produkte centriranih vektorjev in jih deli z M, skladno z enačbo (6).
`CalcRMSE(…)`	Izračuna RMSE pravokotnih odmikov $\hat{\boldsymbol{n}}^T \boldsymbol{p}_i + \hat{d}$; zahteva enotsko normalo.
`RunExperiment(…)`	Izvede eno realizacijo šuma, PCA, izračun metrike in po želji izvoz `measured.csv` in `plane_fit.csv`.
`MonteCarloNoise(…)`	Za vsako σ večkrat pokliče `RunExperiment`, izračuna povprečja in standardni odklon ter shrani `monte_carlo_noise.csv`.
`Result`	Združi `angleError`, `rmse` in `centroidError`, da so rezultati ene rekonstrukcije lahko vrnjeni skupaj.
`main()`	Nastavi geometrijo in generator, sproži posamezno rekonstrukcijo za sliko ter Monte Carlo analizo.

```
    for (auto& p : points) p = R * p + t;
}

void AddGausNoise(std::vector<Eigen::Vector3d>& points,
                  double sigma, std::mt19937& gen) {
    std::normal_distribution<double> noise(0.0, sigma);
    for (auto& p : points) {
        p.x() += noise(gen);
        p.y() += noise(gen);
        p.z() += noise(gen);
    }
}
```

Generator `std::mt19937(42)` se ustvari enkrat v `main()`. Seme 42 omogoča ponovljivost zaporedja, njegovo stanje pa se ob nadaljnjih klicih spreminja. V primerjavi z ustvarjanjem novega generatorja pri vsaki iteraciji s tem preprečimo ponavljanje enakega vzorca šuma. Za dosledno rabo C++ bi pri $\sigma = 0$ šum preprosto preskočili (standardni odklon porazdelitve mora biti pozitiven), čeprav v prikazanem teku ta robni primer ni povzročil napake.

5.2 Težišče, kovarianca in PCA

Težišče se izračuna s seštevanjem vektorjev in deljenjem s številom točk. Zunanji produkt centriranega vektorja je zapisan neposredno z množenjem vektorja in njegove transponirane oblike. Za simetrično kovariančno matriko se uporabi namenski lastni reševalnik knjižnice Eigen; njegov prvi lastni vektor pripada najmanjši lastni vrednosti.

```
Eigen::Matrix3d C = Eigen::Matrix3d::Zero();
for (const auto& p : measured) {
    Eigen::Vector3d q = p - centroid;
    C += q * q.transpose();
}
C /= static_cast<double>(measured.size());
```

```
Eigen::SelfAdjointEigenSolver<Eigen::Matrix3d> solver(C);
Eigen::Vector3d normal = solver.eigenvectors().col(0);
double d = -normal.dot(centroid);
```

Ta izbira numeričnega reševalnika je smiselna, ker izkorišča simetrijo matrike in zagotavlja realne lastne vrednosti ter ortonormirane lastne vektorje. Produkcijska različica naj pred uporabo rezultata preveri še `solver.info() == Eigen::Success`.

5.3 Vrednotenje in organizacija ponovitev

Struktura `Result` vsebuje tri napake ene rekonstrukcije. Funkcija `RunExperiment` sprejme nespremenljivi referenčni oblak, vanj ne posega, ampak ga najprej kopira; vsaka ponovitev zato začne z enakimi idealnimi točkami. Parameter `saveData` prepreči, da bi se pri Monte Carlo analizi CSV z 3D-prikazom prepisoval ob vsaki iteraciji. Ključni del izračuna je:

```
const double dot = std::clamp(
    std::abs(trueNormal.dot(normal)), 0.0, 1.0);
const double angle = std::acos(dot) * 180.0 / std::numbers::pi;
const double rmse = CalcRMSE(measured, normal, d);
const double centroidError =
    (centroid - CalcCentroid(reference)).norm();
```

Na višji ravni `MonteCarloNoise` za vsako σ $K = 100$-krat kliče `RunExperiment`. Aritmetično povprečje kotnih napak $\bar{\theta}$ in vzorčni standardni odklon s_θ sta

$$\bar{\theta} = \frac{1}{K}\sum_{k=1}^{K}\theta_k, \qquad s_\theta = \sqrt{\frac{1}{K-1}\sum_{k=1}^{K}(\theta_k - \bar{\theta})^2}. \tag{17}$$

Program za standardni odklon uporabi računsko ekvivalentno obliko s seštevanjem θ_k in θ_k^2. Zaradi možnih napak zaokroževanja izraza za varianco se pred korenjenjem uporabi `std::max(0.0, variance)`. Predstavljen poskus obsega šest konfiguracij po 100 ponovitev, skupaj 600 PCA-ocen. Ista funkcija se lahko uporabi tudi pri prihodnjih raziskavah gostote vzorčenja, pri čemer bi bilo smiselno statistiko združiti v samostojno ponovno uporabno komponento.

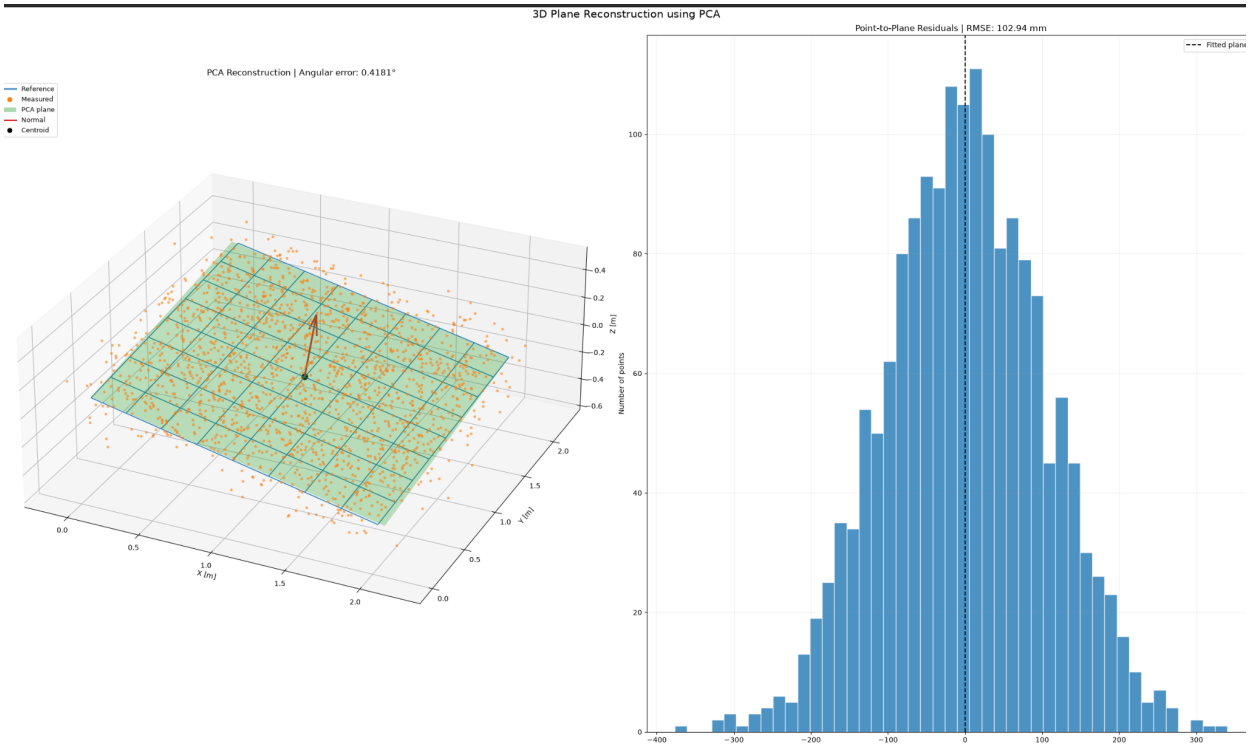
6 Rezultati in razprava

6.1 Posamezna rekonstrukcija

Na sliki 1 se ocenjena in referenčna ravnina skoraj prekrivata, čeprav so izmerjene točke precej razpršene. Histogram je približno simetričen okoli ničle in ima obliko, skladno z dodanim Gaussovim šumom. Razlika med RMSE in kotno napako je bistvena: prvi meri odstopanje zašumljenih meritev od prilegajoče se ravnine, druga pa natančnost ocenjene orientacije glede na znano referenco. Prav zato RMSE približno 103 mm ne pomeni, da se ocenjena ravnina za 103 mm razlikuje od dejanske.

6.2 Monte Carlo analiza vpliva šuma

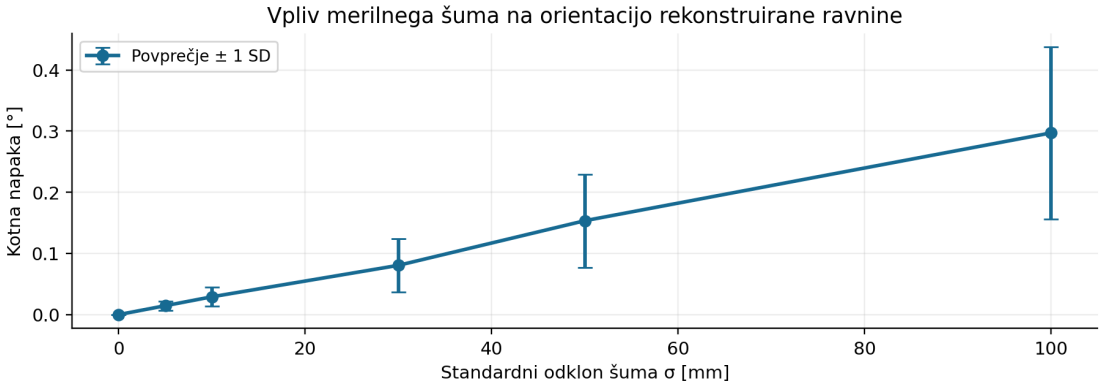
V preglednici 3 so povzeti vsi rezultati 600 ponovitev. Pri nižjih jakostih šuma ostaja kotna napaka zelo majhna, z naraščajočim σ pa se povečuje tako njeno povprečje kot razpršenost med realizacijami. Pri $\sigma = 10\,\text{mm}$ je povprečna kotna napaka $0{,}0291°$, pri desetkrat večjem šumu $\sigma = 100\,\text{mm}$ pa $0{,}2971°$. Znotraj preizkušenega območja je odvisnost približno linearna, kar je skladno z majhnimi motnjami lastnih vektorjev pri nespremenjeni prostorski razsežnosti oblaka. Standardni odklon ob $100\,\text{mm}$ doseže $0{,}1407°$, zato iz enega samega poskusa ni smiselno sklepati na pričakovano točnost metode.



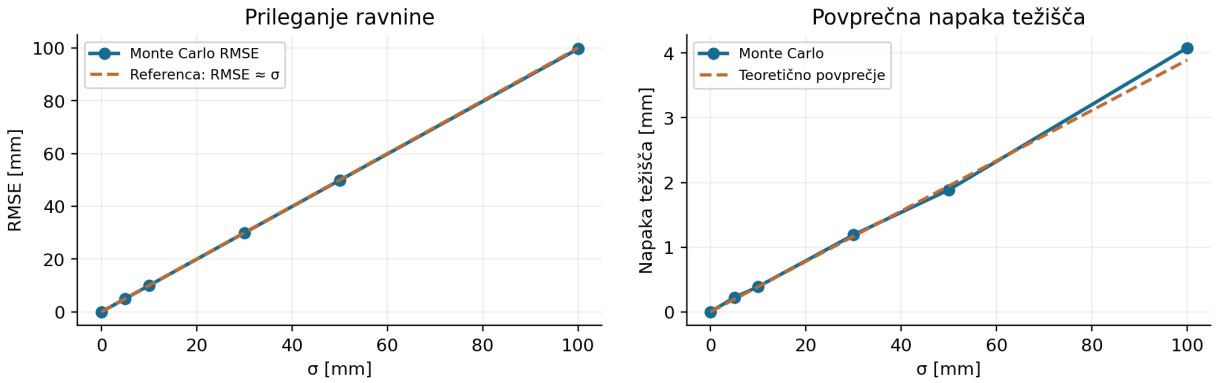
Slika 1. Posamezna rekonstrukcija pri $\sigma = 100\,\text{mm}$: modra mreža prikazuje referenčno ravnino, oranžne točke izmerjeni oblak, zelena površina ocenjeno PCA-ravnino in rdeča puščica normalo. Desno je histogram predznačenih pravokotnih odmikov od ocenjene ravnine. Kotna napaka te realizacije znaša $0{,}4181°$, RMSE pa $102{,}94\,\text{mm}$.

Tabela 3. Rezultati Monte Carlo analize; pri vsaki vrednosti σ je izvedenih 100 rekonstrukcij.

σ [mm]	$\bar{\theta}$ [°]	s_θ [°]	Povpr. RMSE [mm]	Povpr. e_c [mm]
0	0	0	≈ 0	0
5	0,0145	0,0077	5,003	0,221
10	0,0291	0,0152	9,977	0,389
30	0,0805	0,0434	29,943	1,189
50	0,1535	0,0761	49,867	1,886
100	0,2971	0,1407	99,647	4,080



Slika 2. Povprečna kotna napaka rekonstruirane normale v odvisnosti od standardnega odklona šuma. Navpične črte označujejo ± 1 vzorčni standardni odklon, ne intervala zaupanja povprečja.



Slika 3. Levo: povprečni RMSE in referenčna odvisnost RMSE $\approx \sigma$. Desno: povprečna evklidska napaka težišča in teoretična pričakovana vrednost po enačbi (16).

Na sliki 3 je povprečni RMSE skoraj enak vhodnemu standardnemu odklonu šuma: pri $\sigma = 100\,\text{mm}$ znaša $99{,}65\,\text{mm}$. To je pričakovano, saj pri veliki množici točk in izotropnem šumu varianca odmikov vzdolž normale približno ustreza σ^2. Napaka težišča znaša pri isti jakosti šuma $4{,}08\,\text{mm}$, kar se dobro ujema s teoretičnim povprečjem $3{,}89\,\text{mm}$. Povečanje števila neodvisnih meritev zmanjša negotovost težišča približno po zakonu $1/\sqrt{M}$.

Rezultati potrjujejo, da PCA lahko poda natančno oceno *orientacije* tudi pri izrazito zašumljenih posameznih točkah. Vendar analiza temelji na eni sami ravnini, pravilno izbranih pripadajočih točkah in izotropnem šumu. Pri več površinah ali osamelcih kriterij najmanjših kvadratov ni robusten: takrat sta potrebni segmentacija oblaka in robustnejše metode, kot je RANSAC [3]. Poleg tega prikazano Monte Carlo vrednotenje ne zajema realnega kalibracijskega odklona ali od globine odvisnih napak meritev. Analiza vpliva gostote točk in velikosti površine je smiseln naslednji korak, saj na stabilnost normale poleg šuma vplivajo tudi razlike med lastnimi vrednostmi kovariančne matrike.

7 Sklep

Izvedena je bila rekonstrukcija ravninske površine iz zašumljenega oblaka 1681 točk z metodo PCA in implementirana primerljiva Monte Carlo analiza 600 poskusov. Matematična izpeljava pokaže, da je normalo najbolje prilegajoče se ravnine mogoče določiti kot lastni vektor najmanjše lastne vrednosti kovariančne matrike, ker ta smer minimizira kvadratne pravokotne odmike meritev. Numerični rezultati potrjujejo pričakovano rast kotne napake s šumom, bližino RMSE in σ ter zmanjševanje negotovosti težišča s povprečenjem. Opisana implementacija v C++/Eigen predstavlja preverljivo osnovo za nadaljnje raziskave segmentacije ravnin, registracije oblakov točk in 3D-kartiranja.

Literatura

[1] I. T. Jolliffe, *Principal Component Analysis*, 2. izd., Springer, 2002.

[2] G. H. Golub in C. F. Van Loan, *Matrix Computations*, 4. izd., Johns Hopkins University Press, 2013.

[3] M. A. Fischler in R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, 24(6), 381–395, 1981.